

n GitLab.com. Additionally, the ability to isolate customer data is an influencing factor on the design and implementation.

Background

At the onset of this working group, anecdotal information indicated that we were going to "hit a wall" on scaling our database to support our projected customer growth. Early estimates indicated us hitting a scaling wall anywhere from 6 - 12 months down the road. This estimate has since been revised to a rolling 12 month window due to Database Capacity and Saturation Analysis (Iteration 1). Database sharding was proposed as a solution to improve our scalability while simultaneously improving performance. We have since expanded our discussions from solely focusing on database sharding. Any solution, even if using database sharding technology, will require significant application changes as well.

The goal for customer isolation serves multiple purposes. Isolation of customer data would likely include distributing data across multiple servers. This level of distribution would improve availability by removing the single point of failure of our single database architecture. Additionally, we hearing more requests from customers to provide a solution that better separates customer data.

Areas of Investigation

In support of our business goals of scalability and customer isolation we've identified the following areas of investigation.

Namespace Sharding

Details can be found in the Postgres Sharding (&1854) epic. This area of investigation is focuses on sharding at the top-level namespace. The initial investigations were database-centric, focusing on sharding the tables. Our investigations have indicated the following:

- Several application changes would need to happen in coordination with the database changes in order for features to continue to function
 - Identify potential application changes needed to support sharding
- Many tables do not easily lend themselves to namespace sharding
 - Compatible and conflicting features for namespace sharding

Tenant Sharding

A proposal titled Tenant Sharding was recently introduced. Instead of sharding by the namespace, we introduce a higher-level entity, the tenant. By introducing the tenant entity, we turn GitLab.com into a multi-tenant SAAS platform, in the model of SAAS multi-tenant applications. Well known examples include Slack, Pagerduty, Datadog, etc. Each of these examples offers their users a scoped, isolated tenancy.

Incremental Scalability Improvements

In parallel with the sharding investigation, the database team continues to look for areas of incremental database scalability improvements. Those efforts are being tracked under these issues/epics:

- Reduce total size and growth of GitLab.com's PostgreSQL database
- Workload analysis on GitLab.com - first iteration

Database Partitioning Implementations

Partitioning is an important subject to cover separate from sharding. If we ultimately decide that database sharding is the chosen solution to achieve our business objectives, then database partitioning is the foundation upon which database sharding is built in PostgreSQL. Even if we don't use it for sharding, partitioning directly improves query performance and is therefore a great tool to use on its own. Our first iteration of database partitioning will be implemented on audit events. We expect that the implementation of partitioning will result in performance improvements and tooling implementation (e.g. migrations) for subsequent partitioning and sharding implementations.

Investigation Summary

The different sharding approaches, Namespace vs. Tenant, are being evaluated. They are competing approaches but each have the same goal of achieving our business goals. We are still working through the potential first iteration and implementation details of these approaches. In both cases we will need to identify and quantify the changes required at the database and application level.

While we continue to investigate Namespace vs. Tenant sharding, we can continue with the Incremental Scalability Improvements and Database Partitioning Implementation and realize immediate performance and scalability improvements.

Exit Criteria

- x] Infra: [PostgreSQL 11 deployed on GitLab.com - April
- x] Distribution: [Add support for PostgreSQL 11 (13.0)
- Deploy MVC partition (Dependent on PG11 Deployment)
 - Define partition key (may be different than tenancy model for MVC)
- x] [Partitioning spike: Investigate impact on search when partitioning issues
- Implement partitioning MVC Partitioning: Design and implement partitioning strategy for

Audit Events

- Document process to enable backend teams to implement their own partitioning solution going forward
- Measure results
- Investigation
 - x] [Explore CitusDB as a sharding solution
 - x] Document impact of [Sharding GitLab by root namespace
 - Identify application changes required by namespace sharding
 - Review Tenant Sharding approach
 - Identify application changes required by namespace sharding
- Implement sharding strategy
 - Implement and Demonstrate POCs
 - Gather feedback and metrics from POCs
 - Roll out sharding implementation

Roles and Responsibilities

Working Group Role	Person	Title
Executive Stakeholder Development	Christopher Lefelhocz	VP of
Facilitator Manager, Database	Craig Gomes	Engineering
DRI for Sharding Working Group Manager, Database	Craig Gomes	Engineering
Functional Lead Engineer in Test, Database	Nailia Iskhakova	Software
Functional Lead Product Management, Enablement	Josh Lambert	Group Manager,
Functional Lead Fellow, Infrastructure	Gerardo "Gerir" Lopez-Fernandez	Engineering
Functional Lead Fellow, Development	Stan Hu	Engineering
Functional Lead Engineer, Database	Andreas Brandl	Staff Backend
Member Engineering, Enablement	Chun Du	Director of
Member Engineer, Database	Pat Bair	Senior Backend
Member Engineering Manager, Enablement	Tanya Pazitny	Quality
Member Quality Engineering	Mek Stittri	Director of

Meeting Recap

The agenda doc can be found in our Google Drive when searching for "Sharding Working Group Agenda"

- 2020-02-10 - First meeting. Validate attendees, priorities and timing of meetings
 - Blocker identified - We can't ship any changes until PG11 is shipped
 - Blocker identified - We can't use declarative partitioning until completion of Use structure.sql instead of schema.rb
 - Limiting factor - Database team only consists of Andreas plus Pat who joined 2 weeks prior
- 2020-02-19
 - PG11 testing on reference architecture initiated
 - Capacity planning started to determine head room of current architecture
 - Clarity on expectations discussed - performance and scalability
 - Issue created to investigate issues where sharding would have helped
- 2020-02-24
 - Partitioning Issue Spike
 - Handbook entry published On table partitioning
 - PG11 Nightly pipeline for gitlab-org/gitlab
 - PG11 10k Reference Architecture test
- 2020-03-02

- Exploration of CitusDB started
- 2020-03-09
 - Incident Review feedback
 - Handbook entry update to explicitly set priorities for sharding
 - Handbook entry Introducing PostgreSQL table partitioning to GitLab's database
- 2020-03-16
 - Partitioning spike complete
 - Run GDK against a Citus Installation
- 2020-03-23
 - Identify MVC Tenancy Model for Partitioning
 - Unblocked from using PG11 Features when it is available with completion of Use structure.sql instead of schema.rb
 - Partitioning: Investigate increase in planning time (with PG11 and PG12)
 - Yannis Roussos joins Database team
- 2020-03-30
 - Set up Citus Cluster for experimentation
- 2020-04-06
 - Discussions with Legal regarding Citus Community AGPL licensing
 - Discussions with Citus to discuss Enterprise Licensing model
 - Published results for Partitioning: Implement cascading deletes without foreign keys
 - Blocker Identified - Test data for Citus Cluster is a concern
- 2020-04-13
 - Follow up meeting with Citus to discuss licensing model and pricing negotiations
 - Data collection on table sizes to better understand impact of performance and migrations
- 2020-04-20
 - Determined that Citus is a non-starter for community edition due to AGPL and Enterprise due to licensing costs
 - Service Extraction discussions
- 2020-04-27
 - Sharding Brainstorming discussions continue
 - Partitioning/sharding and isolating access patterns
- 2020-05-04
 - Postgres sharding with partitioning and FDW
 - Part 1 - partitioning + FDW
 - Part 2 - Schema migrations
 - Part 3 - Reference tables with logical replication
 - MR WIP / experimental: Postgres partitions + FDW
- 2020-05-11
 - PG11 Shipped to production - we are now unblocked from being able to push MRs to production
 - Focus on Audit Events for partitioning
 - Partitioning: Identify access patterns for Audit Events
- 2020-05-18
 - Capacity assessment for our main production DB cluster
 - Created issue to quantify Conflicting features for namespace sharding
 - Work underway for Audit Events partitioning
 - Migration strategy to partition existing tables: Create partitioned tables
 - Partitioning: Identify access patterns for Audit Events
- 2020-05-26
 - Audit log database design changes
 - Create script to generate test auditevents data

- Compatible and conflicting features for namespace sharding
- 2020-06-01
 - There were a few topics of discussion for this working group
 - Tenant Sharding vs. Namespace Sharding
 - Goals - Availability, Scalability, Customer Isolation
 - We spent some time discussing different implementation approaches
 - Database Capacity and Saturation Analysis (Iteration 1)
 - Snippet: Given 3-month timeframe we have analyzed, we have a high-degree of confidence that the current architecture is in good shape to handle our needs over the next 12 months
 - Progress continues on Partitioning: Design and implement partitioning strategy for Audit Events
- 2020-06-08
 - Working group updates
 - Timeline of Tenant vs. Namespace Sharding
 - Discussions of incremental database changes

SECTION: company/working-groups/simplify-groups-and-projects.md

Attributes

Property	Value
Date Created	May 22, 2020
Date Ended	Oct 30, 2020
Slack	wgsimplify-groups-and-projects (only accessible from within the company)
Google Doc	Working Group Agenda (only accessible from within the company)
Docs/Epics	Epic, Opportunity Canvas
Feature Issue	TBD
Associated OKRs	Increase Stages per Active Namespace for accounts less than 90 days old to >1.7

Problem To Solve

- Groups, sub-groups, and projects as they are organized today are confusing to our users and are negatively impacting adoption, onboarding, and retention for Plan and numerous other stages.
- We currently have a lot of requests to "implement [insert existing project feature here] at the Group level" as a way to try and solve some of this. We are duplicating capabilities that already exist at the Project level but not within the Group.
- When we do this, we end up not cleanly recreating the same experience at the Group level, resulting in defects, missing capabilities, and poor UX that leads to incomplete and painful critical paths for our customers.
- External: GitLab is inflexible to how companies organize their teams and workflows. Internal: We are/will be spending millions of dollars on implementing (or paying down technical debt related to) project level features within Groups (or vice versa).
- See "Pain" and corresponding customer perspective from Opportunity Canvas.

Business Goal

- Instead of this complex organizational model, users can structure GitLab like this simplified organizational model without having to duplicate base functionality (Issues, Wikis, Epics, etc.) between Groups and Projects.
- Increase Stages Per Active Namespace / Stages Per User.
- Improve NPS/SUS scores around usability and performance.

Exit Criteria

(· Done, · · In-progress)

Identify high level vision, MCV, and an iteration plan to solve the problem => 100%

Issue

- Business viability: Does the solution work for our business? ·
- Technical feasibility: Can we build it? ·
- Usability: Can the user figure out how to use it and is it better received than our current approach to Groups and Projects? ·
- Value: Will this drive an increase in key business metrics such as SpU, SPAN, SMAU, and AMAU? ·

Opportunity Canvas: <<https://docs.google.com/document/d/1a2xuEMAsWzPK-cbwnS9DWh9YHXZLS34Lp7jdetODR4/edit>> (internal)

Roles and Responsibilities

Because of the wide sweeping implications of solving this problem -- and that it spans the entire product -- we would like at least one Member per section and ideally one Member per stage.

Working Group Role	Person	Title
Executive Sponsor	Anoop Dawar	VP Product Management
Facilitator	Justin Farris	Group Product Manager, Plan
Functional Lead	Liam McAndrew	Engineering Manager, Manage:Authentication and Authorization
Functional Lead	Gabe Weaver	Senior Product Manager, Plan
Functional Lead	UX Mike Long	Product Design Manager, Dev:Plan & Dev:Manage
Member	Melissa Ushakov (Manage)	Senior Product Manager, Authentication and Authorization
Member	Alex Pooley (Manage)	Senior Backend Engineer, Authentication and Authorization
Member	Christen Dybenko (Create)	Senior Product Manager, Knowledge
Member	Natalia Tepluhina (Create)	Staff Frontend Engineer, Knowledge
Member	Markus Koller (Create)	Backend Engineer, Knowledge
Member	Keanon O'Keeffe	Senior Product Manager, Plan
Member	Mark Wood	Senior Product Manager, Plan
Member	Donald Cook	Frontend Engineering Manager, Plan

Meetings

Meetings are recorded and publicly available on YouTube in the Simplify Groups and Projects Working Group playlist (TBD).

```
<iframe width="560" height="315"
src="https://www.youtube.com/embed/videoseries?list=PL05JrBw4t0KpTkeryiFRk8aJdk-MgqRq"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture"
allowfullscreen></iframe>
```

SECTION: company/working-groups/single-codebase.md

Attributes

Property	Value
Date Created	March 6, 2019
Date Ended	September 13, 2019
Slack	wgsinglecodebase (only accessible from within the company)
Google Doc	Single Codebase Working Group Agenda (only accessible from within the company)
Epic	Single GitLab Rails Codebase

Business Goal

Increase engineering efficiency by removing the busy work created by working in two similar codebases by merging the GitLab Community Edition and GitLab Enterprise Edition repositories.

Exit Criteria

GitLab is being developed in one Rails repository.

Roles and Responsibilities

Working Group Role	Person	Title
Facilitator	Marin Jankovski	Engineering Manager, Delivery
Backend Lead	Yorick Peterse	Staff Backend Engineer
Frontend Lead	Filipa Lacerda	Senior Frontend Engineer
Member	Phil Hughes	Senior Frontend Engineer
Documentation Lead	Achilleas Pipinellis	Technical Writer
Executive Stakeholder	Christopher Lefelhocz	Senior Director of Development

SECTION: company/working-groups/software-supply-chain-security.md

Attributes

Property	Value
-----	-----

Date Created	2023-03-23
End Date	2025-01-31
Slack	wgsoftwaresupplychainsecurity (only accessible from within the company)
Google Doc	Software Supply Chain Security Working Group Agenda (only accessible from within the company)
Issue Label	WorkingGroup::SSCS

Overview and goals

Software Supply Chain Security (SSCS) is a key area where GitLab needs to excel, not only for the benefit of the company but for the benefit and security of our users. SSCS features and capabilities span across nearly all of the stages and groups in GitLab. The primary focus of this working group will be to work on building signing and verification into GitLab and will involve contributions in areas the following groups maintain: Create::Source Code, Verify::Pipeline Execution, Verify::Runner, Package::Package Registry, and Package::Container Registry. There may be some minor code contributions into other team areas as well. The other SSCS feature work at GitLab that is unrelated to signing and verification is outside the scope of this Working Group (but is still being considered or worked on in other areas in GitLab). This direction page describes the broad, long-term future vision for SSCS across the entire SDLC.

Signing and verification are traditionally difficult tasks. To securely sign a build artifact, container image, or package, users need to first securely store a private signing key. This key needs to be regularly rotated, and the signing tool needs a way to securely access the signing key. The combination of all of these requirements result in a high barrier to entry for users to sign their builds. As a result, many private projects and small open source projects do not publish signed build artifacts.

At a high level, this working group plans on making it possible for signing to happen automatically and by default anytime GitLab is used to build an artifact, container image, or package. By taking this complexity away from the end user and enabling it by default, GitLab intends to help lower this barrier to entry and make signing an industry-wide on-by-default standard.

Useful references and links

Some of these links may provide useful background for the working group:

- SLSA framework requirements
- OpenSSF's SLSA verifier
- Sigstore Cosign project

- Sigstore Fulcio project
- Sigstore GitSign project
- WIP engineering blueprint (this is not finalized yet and will require input from the members of this working group)

Anticipated timeline

This working group is anticipated to require a total of 12-15 months to complete. This timeline is only an estimate and the working group will formally conclude once the exit criteria is met. This may result in either a longer or shorter timeline than the estimate of 12-15 months. The group will hold a review at the end of each quarter.

Member commitments and expectations

Unlike other working groups, this effort is expected to take priority over other work. Core members from the Govern stage in the working group should expect to contribute approximately 80% of their time towards this project. Core members from other stages are volunteers and the percentage of their time may be much smaller. This commitment should be discussed and agreed upon with the individual's manager, and product counterpart where applicable, prior to joining the working group. Team members can also join as regular working group contributors.

We intend to re-evaluate the staffing needs of this working group on a quarterly basis. The product and engineering DRIs for the working group may increase, decrease, or swap out team members each quarter based on the overall interests of the team and the needs of the working group. The following dates are planned for re-evaluating the team structure:

- March 2023 - initial formation of group
- 15.10 Milestone (March 17, 2023) - working group officially begins
- July 2023 - end of quarter evaluation of team structure
- 16.2 Milestone (July 17, 2023) - any team changes go into effect
- October 2023 - end of quarter evaluation of team structure
- 16.5 Milestone (October 17, 2023) - any team changes go into effect
- January 2024 - end of quarter evaluation of team structure
- 16.8 Milestone (January 17, 2024) - any team changes go into effect
- April 2024 - end of quarter evaluation of team structure
- 16.11 Milestone (April 17, 2024) - any team changes go into effect

Code management and handoff

Because software supply chain security impacts areas across GitLab, this working group will operate similar to a community contributor by adding functionality into other stages and groups in GitLab. As that functionality is incrementally completed, it will be handed off to the appropriate stage/group for ongoing maintenance.

Exit criteria

The exit criteria for this working group is the completion of the following four criteria:

1. Builds are signed by default, including build artifacts, attestation files, container images, and packages.

1. Commits are signed by default, including commits made through the Web UI. Our approach for doing this commit signing is still being determined.

1. SLSA level 3 attestations are generated by default (currently we generate SLSA level 1 attestations by default)

1. Build signatures and validation status is visible in the UI for build artifacts, container images, and packages. Verification status should be visible by default without requiring users to run custom tooling outside of just viewing the item in GitLab.

Epics to track the above work are still being created, along with the necessary requirements and designs. Please consider the items that do have links as a work-in-progress. Epics to track items that do not have links will be created and added in the future.

Closing Summary

With the Sec section realignment the Govern stage was renamed to the Software Supply Chain Security Stage. With this change we are closing this working group and distributing the work to individual groups to own.

What we achieved:

1. Added GitLab.com as an OIDC provider for Cosign

1. Released a User Experience for Signed Container Registry Images

1. Added documentation on how to Use Sigstore for keyless signing and verification

1. Released storing a source value for Jobs including policies

Priorities and progress

A prioritized list of work will be tracked here after an engineering DRI has been identified.

{{< product/product-priorities software-supply-chain-security-wg >}}

This list is a WIP and we are still working to add a list of prioritized Epics to deliver on the rest of the exit criteria for the group.

Outcome

To be added once the project is complete

Roles and responsibilities

Working Group Role	Person	Title	
Executive Sponsor	Hillary Benson	Senior Director, Product Management	
Product DRI	Sam White	Group Manager, Product - Govern	
Engineering Manager DRI	Nathan Rosandich	Engineering Manager, Govern	
Engineering DRI	Aaron Huntsman	Sr. Backend Engineer, Govern	
Member	Charlie Ablett	Staff Backend Engineer, Plan	
Member	Georgi N. Georgiev	Senior Backend Engineer, Verify	
Advisor	Greg Myers	Security Engineer, Application Security	
Advisor	Ottilia Westerlund	Security Engineer, Application Security	

|

SECTION: company/working-groups/sox.md

Attributes

Property Value
----- -----
Date Created March 11, 2019
Date Ended TBD
Slack wgSOX- PMO (only accessible from within the company)
Google Doc SOX Working Group Agenda (only accessible from within the company)

Business Goals

Project Phase Estimated Completion Timeline Actual Completion Timeline
--- --- ---
Preliminary risk assessment March - April 2019
Creation of SOPs (Detailed plan to be prepared) April 2019 onwards
SOX 404 documentation of process flows/control objectivees for business segments, including key control identification and test of design In sync with SOP creation
SOX 302 certification process Q3 2019
Development of aduit programs and evaluation of operating effectiveness of controls for FY 2020 April/May 2020
Remediation & re-testing Prior to IPO

Exit Criteria

Coming soon....

Roles and Responsibilities

Role Responsibility
----- -----

Facilitator Sheetal Jain
Functional Lead Cindy Nunez, Melody Gallagher
Member Ramakrishnan Hariharan
Executive Stakeholder Paul Machle

SECTION: company/working-groups/teamops-sales-marketing.md

Attributes

Property Value

:----- :-----	
Date Created	2023-03-06
Target End Date	2023-04-31
Overview	Issue
Slack channel	wgteamops
Google Doc	Working Group Agenda (only accessible from within the company)

Goal

The TeamOps Sales and Marketing Working Group aims to manage the design and kickoff of relevant strategies.

Overview

TeamOps needs to generate revenue, and requires a cross-functional working group to discuss, and design one more relevant plans.

Please see the overview issue for more information.

Exit Criteria

1. Prioritize target audiences for TeamOps sales, based on existing product sales and marketing goals
1. Confirm start-to-finish viability of TeamOps B2C sales funnel
1. Confirm lean marketing strategies for TeamOps in FY24
1. Create 1-year partnership plan
1. Brainstorm sales funnel for TeamOps B2B partnerships
1. Define Q2 scope for work for consultant

Roles and Responsibilities

Working Group Role	Username	Person
Title		
:----- :-----		
:-----		
Executive Stakeholder	@streas	Stella Treas Chief of Staff to CEO
Facilitator	@lfarrer	Laurel Farrer Principal Strategy and Operations, Workplace
Member	@cynthia	Cynthia "Arty" Ng Staff Strategy and Operations, Intern
Member	@lboughner	Lisa Boughner VP, Corporate Communications
Member	@dsteer	Dave Steer VP, Product Marketing
Member	@swalters1	Stephen Walters Principal Solutions Architect
Member	@esalvadorp	Emilio Salvador VP, Dev Relations and Community
Member	@nshah19	Niyati Shah Senior Director, Education Services

SECTION: company/working-groups/tiering.md

Attributes

Property	Value
Date Created	October 29, 2020
Date Ended	February 2, 2021
Slack	wgtierin (only accessible from within the company)
Google Doc	Tiering Working Group Agenda (only accessible from within the company)

Business Goal

Leverage team members from other groups to start working on tiering projects so that we can accelerate the delivery of the work.

Roles and Responsibilities

Working Group Role	Person	Title
Facilitator	Justin Farris	Group Manager, Product - Fulfillment
Executive Stakeholder	Sid Sijbrandij	CEO
Executive Stakeholder	Michael McBride	CRO
Member	Xiaohe Li	Principal Pricing Manager
Member	Bartek Marnane	Director of Engineering, Growth
Member	Jack Brennan	Director, Sales Systems
Member	Amanda Rueda	Product Manager, Fulfillment
Member	Dale Brown	Principal Accounting Officer
Member	Christopher Lefelhocz	VP of Development
Member	Scott Williamson	Chief Product Officer

Exit Criteria

The Tiering Working Group will deliver:

- [x] Execute internal transfers
- [x] Onboard transferred team members
- [x] Agree on the scope for the announcement
- [x] Confirm target announcement date
- [x] Launch tiering project

SECTION: company/working-groups/token-management.md

Attributes

Property	Value
Date Created	August 16, 2022
Date Ended	April 25, 2023
Slack	wgtoken (only accessible from within the company)
Google Doc	Token Management Working Group Agenda (only accessible from within the company)

Direction

Out of due diligence and responsibility to GitLab users, the Token Management Working Group will

work towards building a foundation and path forward for future token management security enhancements. This will be accomplished through the creation and publication of a token management

security policy and proposed fixes for the high risk and low effort token management issues. With

this, the Token Management Working Group will set up the fast follow on mid to long term token management security enhancement effort for success. For additional detail please visit our Token Leaks internal handbook page.

All the tokens will be stored in GitLab the application as we ship it to users and customers.

By

dogfooding these enhancements and making improvements to meet our own security needs, we will improve token management and protection of secrets for all users, becoming better stewards of our user's secrets.

Exit Criteria

The Token Management Working Group will deliver:

- Publish the first iteration of an official token and secrets management policy that is based on the Token Standard previously created
 - Status: Complete
 - Results: GitLab Token Management Standard
- Proposed fixes, with risk assessments, for each identified low effort high risk item
 - Status: Complete
 - Results: Risk assessments and effort estimations
- Propose possible out-of-product workaround mitigations for the top 2 high effort high risk items
 - Status: Complete
 - Results: Proposed out-of-product workarounds

Retrospective

The retrospective for the Token Working Group was conducted on 2023-04-25 and details can be found in the retro issue.

Roles and Responsibilities

Working Group Role	Person	Title
Facilitator	James Ritchey	Sr. Manager, Product Security
Member	Joaquin Fuentes	Director Security Operations
Member	Valentine Mairé	SIRT Manager
Member	Philippe Lafoucrière	Security Architect
Member	Andrew Kelly	AppSec Manager
Member	Chris Moberly	Red Team Manager
Member	Connor Gilbert	Sr. Product Manager, Secure:Static Analysis
Member	Grzegorz Bizon	Principal Engineer, Ops
Member	Stan Hu	Engineering Fellow
Member	Michelle Gill	Senior Engineering Manager, Manage
Member	Hannah Sutor	Senior Product Manager, Manage:Auth
Member	Alex Hanselka	Senior Site Reliability Engineer
Member	Dominic Couture	Staff Security Engineer, Application Security
Member	Thomas Woodham	Senior Engineering Manager, Secure
Member	Amar Patel	Engineering Manager, Secure:Static Analysis
Member	Zach Rice	Senior Backend Engineer, Secure:Static Analysis
Member	Lucas Charles	Staff Backend Engineer, Secure:Static Analysis
Member	Dennis Appelt	Staff Security Engineer, Security Research
Member	Mark Loveless	Staff Security Engineer, Security Research

SECTION: company/working-groups/transient-bugs.md

Attributes

Property	Value
Date Created	November 16, 2020
Date Ended	February 23, 2021
Slack	wgtransient-bugs (only accessible from within the company)
Google Doc	Working Group Agenda (only accessible from within the company)
Task Board	Issue board
Bug Board	bug::transient board

Business Goal

Build tooling for transient issues and outline process to sustainably prevent transient issues early on.

Transient bugs have become our focus in FY21Q4 OKRs. We have multiple cross-functional efforts and KR's in both Development and Quality and want to shore-up the momentum on addressing these issues earlier with the appropriate tooling and process in place.

Exit Criteria (100% progress)

1. Reword Engineering FY21Q4 OKRs to reflect proactive mitigation of transient issues. => 100%
1. Build transient developer tooling to help reproduce transient bugs locally. => 100%
1. Build transient test pipeline to help detect transient bugs (GDK,CI,Test flaky reports). => 100%, completed
1. Identify top 3 product groups where fixing transient bugs can help increase GMAU. => 100%, analysis done, need to add to handbook
1. Update our documentation on architectural patterns that can prevent transient bugs. => 100%
 - <https://docs.gitlab.com/ee/development/transient/prevention-patterns.html>
1. Incorporate transient bugs into existing triage process and prioritization. => 100%, have minimal prioritization, working on triage automation
 - <https://handbook.gitlab.com/handbook/engineering/infrastructure/engineering-productivity/issue-triage/merge-requests-experience>
1. Build a measurement to increase visibility of transient bugs. => 100%, there is now a transient bug dashboard in Sisense

Roles and Responsibilities

Working Group	Role	Person	Title
Executive Sponsor		Christopher Lefelhocz	VP of Development
Facilitator		Mek Stittri	Director of Quality
Functional Lead		Tim Zallmann	Director of Development, Dev
Functional Lead		Ramya Authappan	Quality Engineering Manager, Dev
Functional Lead		Valerie Karnes	Director of Product Design
Member		Tanya Pazitny	Quality Engineering Manager, Enablement
Member		Darva Satcher	Senior Engineering Manager, Create
Member		Michelle Gill	Engineering Manager, Create:Source Code
Member		André Luís	Engineering Manager, Create:Source Code, Create:Code Review
Member		Phil Hughes	Staff Frontend Engineer, Create:Code Review
Member		Mark Lapierre	Senior Software Engineer in Test, Create:Source Code
Member		Sofia Vistas	Software Engineer in Test, Package:Package
Member		Erick Banks	Senior Software Engineer in Test, Enablement:Search
Member		Tiffany Rea	Software Engineer in Test, Verify:Continuous Integration

SECTION: company/working-groups/upgrade-improvements.md

Attributes

```
<table>
<tr>
<th>Property</th>
<th>Value</th>
```



```
</tr>
<tr>
<td>Date Created</td>
<td>2024-10-01</td>
</tr>
<tr>
<td>Target End Date</td>
<td>TBD</td>
</tr>
<tr>
<td>Slack</td>
<td>
```

```
wgupgradeimprovements (only accessible from within the company)
</td>
</tr>
<tr>
<td>Google Doc</td>
<td>
```

Working Group Agenda (only accessible from within the company)

```
Project Plan (only accessible from within the company)
</td>
</tr>
<tr>
<td>Epic</td>
<td>
```

```
GitLab self-managed upgrade improvements
</td>
</tr>
<tr>
<td>Label</td>
<td>
```

```
WorkingGroup::UpgradeImprovements
</td>
</tr>
<tr>
<td>Status</td>
<td>Paused</td>
</tr>
</table>
```

Goals

The Upgrade Improvements Working Group is a cross-functional group focused on Upgrade Improvements project, one of the top CTO priorities. The project aims to enhance customer's experience of the entire lifecycle of upgrading a self-managed GitLab instance.

The specific high level goals

1. Expand upgrade cadence options for GitLab users (ie. LTS)
2. Organizational changes in code development
3. Decrease operational requirements of upgrading GitLab
4. Appoint end-to-end owner of GitLab release

Other goals

1. Remove required upgrade stops from GitLab upgrade cadence
2. Breaking changes are limited to major releases

Overview

Upgrading GitLab can be challenging, varying in difficulty from upgrade to upgrade and deployment. The working group reviewed FY25 emergency upgrade tickets so far, gitlab.com deployment blocker dashboard, and customer conversations, group Distribution is also reviewing and validating current upgrading process.

We have found that the current upgrade process itself works in general, there are user experience improvements that can be made to lower the barrier of upgrades, but the upgrade process itself works as intended today. The main challenges are product bugs, configuration issues, migration bug and migration taking too long, these problems are largely existing in the pre- and post- upgrade phases.

Hence, the working group decided to address these concerns in multiple phases.

Phase	Focus areas	ETA
Phase 1		

Database improvements
Code quality
Introduce Support Spikes process
Perform UX research

	FY25 Q4
Phase 2	

Provide option for long term support (LTS) versions

TBD

</td>
<td>TBD</td>
</tr>
<tr>
<td>Phase X</td>
<td>

TBD

</td>
<td>TBD</td>
</tr>
</table>

Refer to Upgrade Improvements project plan (Internal only) and the project Epic for more details.

Exit Criteria

The north star of this working group is increasing the number of self-managed GitLab instances running on maintained versions. However, given that metrics can be influenced by various factors, and some of them are beyond our control, specific exit criteria for each phase are outlined below:

<table>
<tr>
<th>Phase</th>
<th>Exit Criteria</th>
</tr>
<tr>
<td>Phase 1</td>
<td>

Decrease number of self-managed emergency upgrade support tickets relating to database migration

TBD

</td>
</tr>
<tr>
<td>Phase 2</td>
<td>

TBD

</td>
</tr>
<tr>

<td>Phase X</td>
<td>

TBD

</td>
</tr>
</table>

Roles and Responsibilities

Working Group	Role	Person	Title
Executive Stakeholder		Sabrina Farmer	Chief technology officer (CTO)
Facilitator		Peter Lu	Engineering Manager, Distribution::Deploy
Functional Lead - Product & Distribution		Dilan Orrino	Senior Product Manager Distribution
Functional Lead - Test Platform		Kassandra Svoboda	Engineering Manager, Test Platform
Functional Lead - Support		Brie Carranza	Staff Support Engineer
Functional Lead - Expansion Software Development		Thomas Woodham	Senior Engineering Manager, Secure
Functional Lead - Core Development		Luke Duncalfe	Staff Backend Engineer, Manage:Import and Integrate
Functional Lead - Core Development		Erran Carey	Staff Fullstack Engineer, Create::Editor Extensions
Member		Gerardo Lopez-Fernandez	Engineering Fellow, Infrastructure
Member		Vincy Wilson	Director, Test Platform
Member		Lyle Kozloff	Director, Support Engineering
Member		Stan Hu	Engineering Fellow

SECTION: company/working-groups/upstream-diversity.md

Attributes

Property	Value
Date Created	August 1, 2020
Target Date	October 31, 2020
Slack	wgupstreamdiversity (internal only)
Google Doc	Upstream Diversity Working Group Agenda (internal only)
Epic	WG: Upstream Diversity Epic

Business Goal

GitLab has switched to an outbound-only talent acquisition model in an effort to enhance both the quality of our hiring efforts and the diversity of our team simultaneously. One problem left unsolved by this approach is that our technical hiring efforts are downstream from bottlenecks such as the talent pool of the technology industry in general and the university

system, both of which do not reflect society at large. Our diversity value means we have an ethical imperative to try and create more opportunity in the technology industry for currently underrepresented groups. This is despite the fact that it may not benefit GitLab directly because of the timespans involved, and even if it does it may not be measurable.

Knowledge sharing: This working group will design and launch a program to harness the technical talent of the GitLab team to teach technical skills (programming, networking, security, etc) or other skills (remote working, management etc.) if the curriculum requires that to members of underrepresented groups. To do this we may partner with an existing program that already has students and curriculum.

Infrastructure: Secondly, we may try to see if we can help solve infrastructure issues that are holding some people back (such as access to high-bandwidth connections or computers) through a monetary or equipment donation program.

Exit Criteria

Knowledge sharing => 75%

- Create a list of 25 GitLab team members who want to volunteer as GitLab coaches ·
- Find 10 vendors to reach out to (based on our parameters: locality diverse/contributing to enlarging tech talent pool upstream etc.). And narrow down to 3 vendors to collaborate with ·
- Establish a collaboration with 3 vendors and matching at least 15 GitLab coaches to the curriculum ··

Matching efforts will continue.

- Align and contribute to our existing outreach programs, example: GitLab for Startups and GitLab Open Source and make a proposal for an initiative that contributes to our knowledge sharing goal. For example free licenses for high school students interested in tech/ people switching careers. ·

Infrastructure => 100%

- Explore 10 global hardware initiatives facilitating the usage of tech for under represented groups ·
- Write up a proposal for a donation program with one of the global initiatives with reaching 50 people and present to E-Group ·

(· Done, ·· In-progress)

Roles and Responsibilities

Working Group Role	Person	Title
-----	-----	-----

Facilitator	Roos Takken	Engineering People Business Partner
Member	Candace Byrdsong Williams	Diversity, Inclusion and Belonging
Manager		
Member	Mo Khan	Senior Backend Engineer,

Secure:Composition Analysis		
Member	Christian Couder	Senior Backend Engineer Gitaly
Member	Greg Myers	Support Engineer
Member	Rebecca Spainhower	Support Engineering Manager
Member	Edmond Chan	Senior Solutions Architect
Member	Amy Phillips	Engineering Manager, Delivery
Member	Melissa Ushakov	Sr. Product Manager,
Manage::Authentication and Authorization		
Executive Stakeholder Eric Johnson		Chief Technology Officer

SECTION: company/working-groups/user-journey-map.md

These maps will enable Product, UX, and Dev teams to understand the complete journey of customers and GitLab users as they discover, evaluate, adopt, and integrate our DevSecOps platform. By identifying friction points, opportunities for education, and moments that matter most to users, we will create seamless pathways that increase first orders, accelerate successful onboarding, and strengthen long-term retention across our customer base.

While this working group focuses on creating a structured foundation for journey maps, there are various initiatives ongoing related to the creation of maps that can be found in the UX Backlog User Journey Mapping issue.

Attributes

Property	Value	
-----	-----	
Date Created	April 23, 2025	
Target Date	June 30, 2025	
Slack	ujm-working-group (internal only)	
Google Doc	User Journey Map Working Group Agenda (internal only)	

Business goals

This working group aims to:

- Establish how journey mapping contributes to GitLab's business value, growth, monetization, and the company plan
- Establish a strategic framework for user journey mapping across GitLab
- Build an organizational infrastructure for journey mapping
- Operationalize teams around user journeys
- Enable cross-functional collaboration across user journeys

Business outcomes and exit criteria

- Define key terminology including: User journey, golden journey, journey map, and journey elevation/fidelity => 100%
 - Status: Complete
- Develop guidelines for various levels of journey elevation (macro, mid, micro) => 100%
 - Status: Complete
- Establish templates, methodologies, and best practices for journey documentation => 0%
- Create a centralized system for storing, accessing, and connecting journey maps => 50%
 - Status: We have reviewed various tooling and are further examining how to best utilize Figjam
- Establish communication channels for journey insights and discoveries => 0%
- Create a structure for translating journey insights into strategic opportunities based on business impact and user value => 50%
 - Status: We have drafted an issue template for teams to utilize, currently under review

Roles and Responsibilities

Working Group Role	Person	Title
----- ----- -----		

Facilitator	Taurie Davis	Senior Manager of Product Design
Executive Stakeholder	Valerie Karnes	Director of Product Design
Member	Emily Sybrant	Product Design Manager, Monetization
Member	Paige Cordero	Senior Product Manager, Growth
Member	Jessica Kane	Senior Manager of UX Research.
Member	Ben Leduc-Mills	Staff UX Researcher
Member	John Hope	Senior Manager of Engineering, Core
Development		
Member	Suzanne Selhorn	Staff Technical Writer

SECTION: company/working-groups/vuejs-3-migration.md

Attributes

Property	Value
----- -----	

Date Created	2023-02-10

	Target End Date 2025-12-31
	End Date 2024-10-10
	Slack wgvue3migration (only accessible from within the company)
	Google Doc Agenda (only accessible from within the company)
	Epic gitlab-org/&2652
	Overview & Status See Exit Criteria

Disbanding

On October 10th 2024, the Vue 3 working group was disbanded in favor of smaller, more focused task groups.

On this date, the migration to Vue 3 is not complete and there's still a great deal of work to be done.

Please see the exit criteria for information on what's left to complete.

Context

Vue.js 3 is the latest version of the frontend framework used by GitLab.

With 2023-12-31 Vue.js 2 (which is currently the version used by GitLab) reaches official end-of-life, including security updates and browser compatibility which might affect entire GitLab users.

Additionally, current GitLab codebase relies on subset of libraries which were not updated to be compatible with Vue 3 which might pose additional risks during migration.

We want to ensure that we are not creating additional security risks for our users by using an unmaintained version of a framework. At the same time increasing developer productivity by using new features of the latest version of the framework. Also an upgrade to Vue.js 3 hopefully will result in performance gains through out the application of 20-30% and most probably even more for heavy frontend applications like Code Review for example.

Goals

This Working Group has the following goals:

1. Ensure all GitLab projects (including, but not limited to GitLab.com, customers portal, status page, etc.) are using latest Vue.js version
1. Develop & Socialize upgrade strategy for Vue.js 3.
 1. Identify dependencies which are blocking upgrade and ensure that decisions regarding these dependencies will be taken in timely fashion.
 1. Ensure visibility of upgrade process on per-application level
 1. Promote new patterns and update handbook with guidelines appropriate for new framework version

1. Develop a Rollout Strategy and Plan for gradual migration
 1. Create tooling and proper infrastructure to be able to use both framework versions in production during migration process.
 1. Ensure our test suite could be run using both versions of Vue.js to embrace values of iteration of results and allow gradual migration of codebase to Vue.js 3
 1. Implement compatibility layer for unifying differences between framework versions
 1. Create automated tooling (where applicable) to migrate existing Vue.js 2 code to be compatible with latest framework version
 1. Create deprecation strategy for patterns and solutions which are not compatible with Vue.js 3 to rest of the teams
1. Develop and implement a communication plan for the outcomes of the working group.

Exit Criteria

As this group has been disbanded, not all exit criteria has been met.
 The main items have been picked up by related task groups.
 The final two will be picked up once they're unblocked.

Criteria	Start Date	End Date	Progress	DRI	Continued
Get BootstrapVue running with Vue Compat mode 2	2021-06-21	2022-10-25	100%	@xanf	Complete
Get GitLab UI running with Vue Compat mode 2	2021-06-21	-	70%	@markrian	GitLab UI task group
Vue Compat mode on vue-router on GitLab Main	2023-05-18	-	50%	TBD	???
Use Vue.js 3 for running test suites on main repo	2023-05-18	-	30%	@xanf	Company Responsibility
Get CustomersDot running with Vue Compat mode 2	2024-02-01	-	0%	@vitallium	???
Switch Vue Compat from mode 2 to mode 3	-	-	0%	TBD	TBD
Switch to using Vue 3 proper	-	-	0%	TBD	TBD

Roles and Responsibilities

Working Group Role	Person	Title
Executive Sponsor	Tim Zallmann	Senior Director of Engineering
Facilitator	Sam Beckham	Engineering Manager, Manage:Foundations
Functional Lead	Illya Klymov	Senior Frontend Engineer, Govern:Compliance
Functional Lead	Natalia Tepluhina	Principal Engineer, Plan
Functional Lead	Stanislav Lashmanov	Senior Frontend Engineer, Create: Code Review
Functional Lead	Andrew Fontaine	Senior Frontend Engineer, Deploy:Environments
Member	Mark Florian	Staff Frontend Engineer, Foundations:Design System

	Member	Andrei Zubov	Frontend Engineer, Deploy:Environments
	Member	Artur Fedorov	Senior Frontend Engineer, Govern:Security Policies
	Member	Frédéric Caplette	Senior Frontend Engineer, Verify:Pipeline
Authoring	Member	Lukas Eipert	Staff Frontend Engineer, Foundations:Design System
	Member	Marina Mosti	Senior Frontend Engineer, Switchboard
	Member	Nathan Dubord	Senior Frontend Engineer, about.gitlab.com
	Member	Vanessa Otto	Senior Frontend Engineer, Foundations:Design
System	Member	Vitaly Slobodin	Staff Frontend Engineer, Fulfillment

SECTION: company/working-groups/webpack.md

Attributes

Property	Value
----- -----	
Date Created	June 24, 2019
Date Ended	February 20, 2020
Slack	wgwebpack (only accessible from within the company)
Google Doc	Webpack Working Group Agenda (only accessible from within the company)
Epic	Webpack Working Group

Background

GitLab has been using webpack to bundle its frontend assets for three years now, and it has enabled us to do some great things, but at the same time we are not taking full advantage of its strengths (e.g. code splitting and performance improvements) and we are letting it get in the way of developer productivity (high resource consumption, crashes, broken debug tooling, etc) by not adequately addressing issues as we scale. This working group aims to both fix the most glaring issues with our frontend build tooling and to re-align the frontend development guidelines to promote best practices for performance and maintainability using webpack's underutilized features.

Business Goal

Increase engineer productivity by fixing and improving our frontend build tooling, and put both guidelines and automated processes in place to ensure best practices are followed for performance and maintainability.

Exit Criteria

- Improve GitLab's development environment.
 - Reduce the GDK's node process overall memory requirements 30% by improving the webpack/sprockets build process, and put measures in place to track this consumption over time, ensuring it does not grow unchecked. => Reduced by 22% so far
- Improve GitLab's overall frontend performance.
 - Implement targeted builds for modern and legacy browsers (defined as those which do and do not support `<script type="module">` respectively) with only the code transformations and polyfills needed by each target.
 - Document code-splitting policies and put together a training workshop for frontend engineers and upload to youtube.
 - Use webpack output stats to drive performance improvements
 - Determine which webpack output stats should be tracked and monitored (entrypoint bundle sizes, number of entrypoints, initial page load code coverage, duplicate modules across bundles)
 - Implement measures to track webpack output stats. (gitlab-ce!31537)
 - Prescribe limits to these metrics, and goals for reducing them.
 - Put CI jobs in place to warn developers when these increase and enforce best practices.

Conclusion

This effort is now looked after by the Manage:Foundations group.

Roles and Responsibilities

Working Group Role	Person	Title
Frontend Lead	Mike Greiling	Senior Frontend Engineer
Facilitator	Lukas 'Eipi' Eipert	Senior Frontend Engineer
Member	Jake Burden	Frontend Engineer
Member	Tristan Read	Senior Frontend Engineer
Member	David 'DJ' Mountney	Senior Distribution Engineer
Executive Stakeholder	Christopher Lefelhocz	VP of Development

SECTION: company/working-groups/database-scalability/_index.md

Attributes

Property	Value
Date Created	March 15, 2021
End Date	July 18, 2022
Slack	wgdatabase-scalability (only accessible from within the company)
Google Doc	Working Group Agenda (only accessible from within the company)
Issue Board	Sharding:Build Board

Exit Criteria

The charter of this working group is to produce a set of blueprints, design and initial

implementation for scaling the database backend storage. These exit criteria will set us on a path to achieve the following high-level goals:

- Accomodate 10M daily-active users (DAU) on GitLab.com
- Do not allow a problem with any given data store to affect all users
- Minimize or eliminate complexity for our self-managed use-case

These entail access, separation, synchronization, and lifecycle management considerations for various use-cases (see Scaling patterns below).

	Start Date	Completed Date	Criteria
--	-----	-----	-----
1	2021-03-25	2021-06-02	Blueprints for database scaling patterns (read-mostly, time-decay, sharding)
2	2021-04-21	2021-06-03	Horizontal sharding evaluation and proof of concept
3	2021-04-26	2021-06-10	Decomposition (vertical sharding) evaluation and proof of concept
4	2021-06-11	2022-07-02	Rollout decomposed CI database on gitlab.com
5	2022-07-05	2022-07-18	Working group closing tasks, communications, cleanup

Closing Summary

We successfully completed the final exit criteria, decomposing the gitlab.com database into main and CI, on 2022-07-02.

After a couple of weeks of monitoring, the site appears stable and there are significant headroom improvements and reductions in saturation. We will continue to track and report on decomposition improvements in the issue CI Decomposition Performance Summary.

Although this working group is closing, our work to improve GitLab's database and overall application scalability is not finished. Some of our upcoming priorities that will follow up on the work done in this group include:

- Database Partitioning
- CI Decomposition Follow-up and Decomposition support for self-managed customers
- PoCs and first iterations for GitLab Pods

Glossary

Preferred Term	What Do We Mean	Terms Not To Use	Examples
-----	-----	-----	-----
Case	An instance under a scaling pattern		cibuilds, webhookslogs
Cluster	A database cluster is a collection of interconnected database instances that replicate data.		The PostgreSQL cluster of GitLab.com (managed by Patroni) that hosts the main logical database and consists of the primary database instance along with its read-only replicas.
Decomposition	Feature-owned database tables are on many logical databases on multiple database servers. The application manages various operations (ID generation, rebalancing etc.)		
Y-Axis, Vertical Sharding	All CI tables in a separate logical database.		Design illustration

| Instance| A database instance is comprised of related processes running in the database server. Each instance runs its own set of database processes. | Physical Database | |

| Logical database | A logical database groups database objects logically, like schemas and tables. It is available within a database instance and independent of other logical databases.

| Database | GitLab's rails database. |

| Node | Equivalent to a Database Server in the context of this working group. | Physical Database | |

| Replication | Replication of data with no bias. | X-Axis, Cloning | What we do with our database clusters today.|

| Scaling Pattern | A general solution based on data taxonomy that enables scaling for several cases | Split Pattern | Read-Mostly, Time-Decay, Entity/Service |

| Schema |A database schema is a namespace that contains named database objects such as tables, views, indexes, data types, functions, stored procedures and operators.| | |

| Server | A database server is a physical or virtual system running an operating system that is running one or more database instances. | Physical Database | |

| Sharding | Separation of data by customer or requestor bias, or within a resource without bias towards affinity. Sharding via a sharding key, for example top-level namespace, where data is separated into many logical databases on multiple database servers. The application manages various operations (ID generation, rebalancing etc.)| Z-Axis, horizontal sharding | Each top-level namespace is located in its own logical database. Design illustration |

| Table | A database table is a collection of tuples having a common data structure (the same number of attributes, in the same order, having the same name and type per position) (source) |

| |

| Table Partitioning | A table